PPAPlace: Enhancing Chip Placement via Mixed-Size PPA Prediction

Anonymous Author(s)

Abstract

Macro placement significantly determines the post-route performance, power, and area (PPA) of modern integrated circuits. Most placement methods optimize half-perimeter wirelength (HPWL), a fast estimate of total wire length. However, recent benchmarking reveals near-zero correlation between HPWL and post-route timing, and shows that all evaluated AI methods degrade PPA relative to the hierarchical baseline placements. Recent efforts, such as LaMPlace, have begun to bridge this gap by learning cross-stage metric predictors. However, existing approaches rely on macro-centric representations and pre-route supervision that may not fully reflect post-route outcomes. This paper presents PPAPlace, a placer-agnostic framework built on a mixed-size placement PPA predictor. To identify a reliable and cost-effective source of training labels, comprehensive experiments are conducted across design flow stages. The results reveal that timing performance measurements after global-routing better preserves the final post-route results, with only one-sixth the runtime of the full routing process. Based on these findings, a dual-stream predictor combining graph attention over the netlist with spatial convolution over the placement grid is trained on post-global-routing labels to estimate post-route PPA directly from placement-stage features. This design makes the framework independent of any specific placement algorithm. Evaluated on five unseen ChiPBench circuits through a full RTL-to-signoff flow, PPAPlace improves TNS by 33% and WNS by 18% over the Hier-RTLMP hierarchical baseline, outperforming LaMPlace by 13 and 7 percentage points respectively. All code and data will be publicly available.

Keywords

chip placement, PPA prediction, half-perimeter wirelength, design space exploration, graph attention network, physical design

ACM Reference Format:

Anonymous Author(s). 2026. PPAPlace: Enhancing Chip Placement via Mixed-Size PPA Prediction. In *Proceedings of IEEE/ACM International Conference on Computer-Aided Design (ICCAD'26).* ACM, New York, NY, USA, 9 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Chip placement determines the physical locations of circuit modules on a two-dimensional canvas and is one of the most significant steps in VLSI physical design. The quality of placement directly affects the chip's final performance, power consumption, and area (PPA), which together determine whether a design meets its specifications and can be manufactured [12].

Existing placement approaches fall into two broad categories. Analytical placers such as DREAMPlace [17] formulate placement as continuous optimization of a smooth wirelength approximation plus density penalties, solved by gradient descent on GPU. AI-based methods including reinforcement learning [13, 14, 20], black-box optimization [21], and diffusion models [15] replace or augment the optimizer with learned policies. Despite their differences, both categories overwhelmingly optimize half-perimeter wirelength (HPWL), a fast estimate of total wire length, as the primary placement objective.

However, growing evidence suggests that HPWL is a poor proxy for the metrics that matter in practice. The ChiPBench benchmark [25] evaluated six AI-based placement methods on 20 circuits through the full OpenROAD [10] post-route flow. ChiPBench uses Hier-RTLMP, OpenROAD's built-in hierarchical macro placer, as the baseline. Hier-RTLMP leverages RTL design hierarchy and dataflow structure rather than optimizing wirelength alone, yet every evaluated AI method degraded PPA relative to it. ChiPBench further reported a Pearson correlation of only -0.08 between macro HPWL and worst negative slack (WNS), confirming that wirelength optimization is essentially uninformative for timing.

Recent efforts have begun to address this disconnect from both industry and academia. Commercial tools such as Synopsys DSO.ai [23] and Cadence Cerebrus [2] use reinforcement learning to tune tool configurations on top of existing analytical engines, treating placement as a design-space exploration problem. LaMPlace [5] trains a cross-stage predictor on offline placement data and uses it to guide macro placement through evolutionary search, achieving notable improvements in timing metrics. These efforts demonstrate that learned PPA prediction can meaningfully improve placement quality. However, two open questions remain for the field: what is the most reliable source of supervision for cross-stage prediction, and how much of the placement state does the predictor need to observe.

This paper investigates both questions. To address the first, a systematic study is conducted across chip design flow stages to determine which stage provides training labels that best reflect post-route outcomes. The results reveal that pre-route timing evaluation can be misleading: configurations that appear timing-clean before routing violate timing afterward, and their ranking reverses. Post-global-routing evaluation, by contrast, better preserves the post-route ranking in our experiments at roughly one-sixth the cost of full routing. This finding has implications for any learning-based approach that relies on cross-stage supervision.

To address the second question, a dual-stream predictor is proposed that observes the complete mixed-size placement state rather than macro positions alone. A graph attention network processes

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

ICCAD'26, San Jose, CA, USA

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-XXXX-X/26/11
https://doi.org/XXXXXXX.XXXXXXX

the netlist connectivity, and a spatial convolutional network processes the placement grid, capturing standard cell density, pin distribution, and routing congestion estimates. Because the predictor takes any finished placement as input regardless of how it was produced, it is not tied to a specific placement algorithm. The paper demonstrates its application through Bayesian optimization over DREAMPlace's hyperparameter space, but the same predictor could guide configuration search for any placement method.

The contributions are:

(1) A label fidelity study across design flow stages. Pre-route PPA metrics are unreliable for ranking placement quality, while post-global-routing metrics offer the best trade-off between correlation with final outcomes and evaluation cost.
(2) A PPA predictor over the mixed-size placement state. Unlike macro-centric predictors, it captures standard cell density, pin distribution, and routing congestion through a dual-stream architecture combining graph attention with spatial convolution.
(3) A placer-agnostic optimization framework. The predictor scores any finished placement, enabling candidate selection or configuration search across different placement paradigms.

2 Related Work

2.1 Analytical Placement

Analytical placement has been the backbone of chip placement for decades. Among them, DREAMPlace [17] reformulated analytical placement as a neural network training problem, enabling GPU acceleration with over 30× speedup compared to the CPU-based RePlAce [4]. Later versions added timing-driven net weighting [16] and support for mixed-size macros. Building on DREAMPlace, AutoDMP [1] introduced Bayesian optimization to automatically tune placement hyperparameters, an approach conceptually similar to what commercial tools like Synopsys DSO.ai [23] do at a larger scale. DG-RePlAce [11] showed that exploiting dataflow structure in machine learning accelerators can reduce wirelength by 34%. A common limitation across all these methods is that they optimize HPWL or smooth approximations of it, sharing the HPWL-to-PPA gap that we aim to address.

2.2 AI-Based Placement

On the other hand, AI-based methods have explored fundamentally different placement strategies. Despite the controversial reproducibility, AlphaChip [20] pioneered deep reinforcement learning for macro placement [9, 19]. MaskPlace [14] treated placement as visual representation learning, and ChiPFormer [13] improved sample efficiency through offline RL. WireMask-BBO [21] showed that evolutionary algorithms with appropriate problem formulations can match or outperform RL methods on wirelength. MaskRegulate [22] shifted the paradigm from placing macros from scratch to refining existing placements, and EfficientPlace [6] combined tree search with RL to reduce exploration cost. Despite the diversity of these approaches, they all optimize HPWL or macro HPWL as the primary objective.

2.3 Cross-Stage PPA Prediction and Optimization

LaMPlace [5] is a representative recent work that directly targets the HPWL-to-PPA gap. It trains a Laurent polynomial predictor on offline data to estimate cross-stage metrics from macro pairwise distances and uses the predictor to generate a guidance mask for WireMask-EA's evolutionary search, achieving 43% and 30% improvements in TNS and WNS respectively. Its training labels are obtained through DREAMPlace cell placement followed by OpenTimer static timing analysis, providing pre-route timing supervision. MacroRank [3] trained a model to rank macro placements by routing quality but did not integrate it into a placement optimizer. PreRoutGNN [26] predicted pre-routing timing with high accuracy but focused on prediction rather than using the predictions to improve placement. In the commercial space, Synopsys DSO.ai [23] and Cadence Cerebrus [2] use reinforcement learning to tune tool configuration parameters on top of existing analytical engines, treating placement as a design-space exploration problem rather than modifying the placement objective itself.

2.4 Benchmarking

Traditional placement benchmarks such as ISPD 2005 and ICCAD 2004 evaluate methods using wirelength and density metrics alone, without considering post-route PPA. ChiPBench [25] addressed this limitation by providing 20 circuits across CPUs, GPUs, and NPUs with a complete OpenROAD-based evaluation flow that measures post-route WNS, TNS, power, area, and DRC violations. The finding that every evaluated AI method degrades PPA relative to the hierarchical baseline directly motivates our work. We adopt ChiPBench's evaluation protocol as our primary benchmark.

3 Preliminaries

3.1 Chip Placement and Evaluation Metrics

Chip placement assigns physical positions to circuit modules on a two-dimensional canvas. A netlist hypergraph $H = (V, E)$ specifies the modules V (macros and standard cells) and the nets E connecting them. Placement methods seek positions $\mathbf{x} = \{(x_i, y_i)\}_{i=1}^{N}$ that minimize a placement objective. In analytical placers such as DREAMPlace [17], this takes the form:

$$\min_{\mathbf{x}} \ \mathcal{L}(\mathbf{x}) = \mathcal{W}(\mathbf{x}) + \lambda \cdot \mathcal{D}(\mathbf{x}) \tag{1}$$

where $\mathcal{W}$ is a smooth wirelength approximation and $\mathcal{D}$ is a density penalty that discourages cell overlap [18]. The density weight λ increases iteratively to enforce legality. The wirelength term is typically based on HPWL:

$$\mathrm{HPWL}(\mathbf{x}) = \sum_{e \in E} \left[\max_{i \in e} x_i - \min_{i \in e} x_i + \max_{i \in e} y_i - \min_{i \in e} y_i \right] \tag{2}$$

which estimates total wire length by summing the bounding-box half-perimeters across all nets. HPWL is differentiable (via smooth approximations such as the weighted-average model [8]), decomposable across nets, and fast to compute.

However, the actual quality of a placed design is determined by post-route PPA metrics. Given a placement $\mathbf{x}$, let $\mathrm{Flow}(\mathbf{x})$ denote the result of executing the full downstream design flow (cell placement,

Table 1: Spearman rank correlation (ρ) between intermediate-stage timing and post-route timing across 10 ChiPBench circuits, ordered by macro count. Both WNS and TNS correlations are reported. Post-GRT consistently shows strong positive correlation while HPWL and post-CTS are unreliable.

		bp_be (10)	bp_fe (11)	bp_be12 (12)	isa_npu (15)	swerv (28)	vga_lcd (62)	ether (64)	dft68 (68)	mor1kx (78)	ari133 (132)
HPWL	ρ_{WNS}	+.12	−.15	+.08	−.21	+.05	−.18	−.09	+.14	−.11	+.03
	ρ_{TNS}	+.09	−.22	+.15	−.17	−.08	−.13	+.06	+.11	−.19	−.05
Post-CTS	ρ_{WNS}	.83	.64	.71	.38	−.12	.25	−.31	.47	.19	.77
	ρ_{TNS}	.78	.59	.65	.42	−.08	.31	−.25	.38	.15	.71
Post-GRT	ρ_{WNS}	.80	.91	.87	.85	.89	.82	.78	.84	.86	.94
	ρ_{TNS}	.83	.88	.85	.81	.87	.79	.82	.80	.83	.91
Post-DRT	ρ_{WNS}	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00
	ρ_{TNS}	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00
GRT cost (min)		3.2	3.8	3.5	5.1	8.7	12.4	11.8	13.2	14.6	18.3
DRT cost (min)		18.5	22.1	19.8	31.4	52.3	74.8	71.2	78.5	86.4	108.7

CTS, routing, and timing analysis). The PPA outcome is:

$$\mathbf{y} = \text{Flow}(\mathbf{x}) = [\text{WNS},\ \text{TNS},\ \text{Power},\ \text{Area}] \tag{3}$$

where WNS is the worst negative slack (the most critical timing violation), TNS is the total negative slack (sum of all violations), and power and area are the total power consumption and physical footprint. Computing $\mathbf{y}$ requires running the full design flow, which typically takes tens of minutes to hours per design.

3.2 Design Flow Stages

The design flow between placement and signoff consists of several stages, each providing increasingly accurate but more expensive evaluation. Let $\text{Flow}_s(\mathbf{x})$ denote a partial flow truncated at stage s:

$$\mathbf{y}_s = \text{Flow}_s(\mathbf{x}), \quad s \in \{\text{CTS},\ \text{GRT},\ \text{DRT}\} \tag{4}$$

- **CTS** ($\mathbf{y}_{\text{CTS}}$): constructs a balanced clock distribution network that delivers the clock signal to all sequential elements, inserting buffers to meet setup timing constraints.
- **GRT** ($\mathbf{y}_{\text{GRT}}$): assigns nets to coarse routing regions across the chip, introducing congestion-dependent wire delay estimates.
- **DRT** ($\mathbf{y}_{\text{DRT}}$): assigns exact metal tracks and vias for every net, producing signoff-quality timing, power, and area metrics.

The goal of a cross-stage PPA predictor is to learn $f_\theta(\mathbf{x}) \approx \mathbf{y}_{\text{DRT}}$ from placement-stage features, avoiding the cost of running $\text{Flow}(\mathbf{x})$ at inference time. A prerequisite is that the training labels $\mathbf{y}_s$ used to supervise f_θ must rank placements consistently with the final outcome $\mathbf{y}_{\text{DRT}}$. Section 4 investigates which stage s best satisfies this requirement.

4 Label Fidelity Analysis

Any learning-based PPA predictor requires training labels that rank placements consistently with their final post-route quality. This section investigates which stage offers the best trade-off between ranking fidelity and evaluation cost across a diverse set of circuits.

4.1 Setup

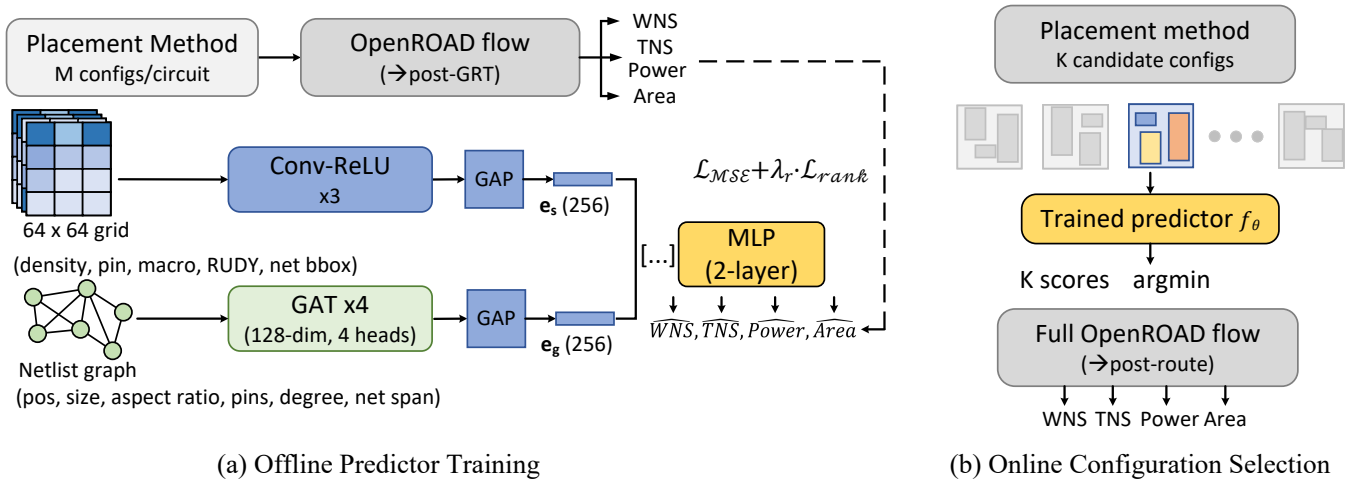
The study spans 10 ChiPBench [25] circuits covering RISC-V CPUs from three families (bp_fe, bp_be, and bp_be12 from BlackParrot, swerv_wrapper from SweRV, ariane133 from Ariane), an OpenRISC CPU (mor1kx), a neural processing unit (isa_npu), and peripheral designs (ethernet, dft68, vga_lcd). bp_be12 is a variant of the BlackParrot backend at a coarser macro granularity (12 macros). Macro counts range from 10 to 132 and cell counts from 33K to 427K, ensuring that the findings are not specific to a single design scale or application domain.

To generate diverse placements for each circuit, the weight parameters of OpenROAD's RTLMP hierarchical macro placer [10] are systematically varied. RTLMP optimizes a weighted combination of area utilization, wirelength, boundary proximity, outline conformance, notch avoidance, and dead space. By sweeping these weights, the same deterministic placer produces structurally different macro arrangements while remaining within the space of physically reasonable placements. For each circuit, 20 configurations are generated, including single-weight extremes (e.g., high or low area weight), pairwise combinations (e.g., high wirelength and boundary weights), and globally scaled settings.

Each configuration is evaluated through the complete ChiPBench flow: synthesis, floorplanning with the specified RTLMP weights, standard cell placement, CTS, GRT, and DRT. PPA metrics (WNS, TNS, power) are recorded at each stage. Area is excluded as it is determined by the floorplan and does not vary across configurations of the same circuit. Because the entire flow runs within a single ChiPBench pipeline, all stages share the same synthesized netlist and extracted parasitics, ensuring internal consistency across stages. Post-DRT metrics serve as the ground truth throughout this analysis.

4.2 Stage-Wise Rank Correlation

Table 1 reports the Spearman ρ between WNS at each intermediate stage and the post-route ground truth, computed over the same set of placements per circuit.



(a) Offline Predictor Training (b) Online Configuration Selection

Figure 1: PPAPlace framework. (a) Offline: diverse placement configurations are evaluated through the OpenROAD flow to post-global-routing. A dual-stream predictor (GAT + CNN → MLP) is trained on the resulting (placement, PPA) pairs. (b) Online: the predictor scores K candidate placements produced by any placement method. Only the top-ranked candidate proceeds through the full post-route flow.

Circuits are ordered by macro count (10 to 132). HPWL shows near-zero correlation with post-route WNS across all circuits ($|\rho| < 0.22$), confirming its unreliability as a timing proxy. Post-CTS correlation is inconsistent: strongly positive for some circuits (bp_be: $\rho = 0.83$) but negative for others (ethernet: $\rho = -0.31$; swerv: $\rho = -0.12$). Post-GRT achieves consistently high correlation across all 10 circuits ($\rho \geq 0.78$), with no circuit showing the sign reversals observed at earlier stages. The pattern holds from the smallest design (bp_be, 10 macros, $\rho = 0.80$) to the largest (ariane133, 132 macros, $\rho = 0.94$). GRT evaluation costs roughly one-sixth of full DRT across all circuits tested.

4.3 Implications

The consistent pattern across 10 circuits spanning 10 to 132 macros, 33K to 427K cells, and five design categories (CPUs, NPU, networking, DFT, display) demonstrates that post-global-routing is the most cost-effective label stage for cross-stage PPA prediction. Post-GRT evaluation achieves consistently high rank correlation with post-route outcomes at roughly one-sixth the runtime of full detailed routing. Labels from stages before global routing, including the pre-route static timing used by existing methods [5], fall in the range where ranking disagreement with post-route outcomes is observed. While the severity may vary across technology nodes and design styles, these findings suggest that validation against post-route outcomes is necessary when choosing a label source for cross-stage prediction.

5 PPAPlace

Based on the findings in Section 4, PPAPlace uses post-global-routing labels as training supervision. The framework consists of a PPA predictor trained on the mixed-size placement state (Sections 5.1–5.3) and a placer-agnostic configuration optimization mechanism that uses the predictor to guide any placement method (Section 5.4). Figure 1 illustrates the overall framework.

5.1 Mixed-Size Placement Representation

Existing cross-stage predictors such as LaMPlace [5] represent the placement state through macro pairwise distances, discarding information about standard cells, spatial density, and routing congestion. However, timing violations and power consumption are largely determined by standard cell placement and routing, not macro positions alone [25]. PPAPlace addresses this by observing the complete mixed-size placement through two complementary representations.

Spatial representation. The placement canvas is rasterized into a 64×64 grid with five channels:

(1) Cell density: total cell area in each bin normalized by bin capacity.
(2) Pin density: number of I/O pins per bin.
(3) Macro occupancy: binary mask indicating macro presence.
(4) RUDY congestion: estimated routing demand per bin [7], computed from net bounding boxes.
(5) Net bounding box density: number of net bounding boxes overlapping each bin, capturing routing pressure from nets whose pins lie outside the bin.

All channels are normalized to $[0, 1]$. This grid-based representation captures spatial distribution patterns invisible to macro-only predictors, including congestion hotspots and density imbalances.

Graph representation. The netlist is represented as a graph $G = (V_G, E_G)$ where nodes correspond to macros. Edges are derived from the netlist hypergraph by star decomposition: each multi-pin net produces a clique of edges among its connected macros. Each node carries a feature vector:

$$\mathbf{h}_i = [x_i,\ y_i,\ w_i,\ h_i,\ r_i,\ p_i,\ n_i,\ s_i] \tag{5}$$

where (x_i, y_i) is normalized position, (w_i, h_i) is normalized width and height, $r_i = w_i/h_i$ is aspect ratio, p_i is pin count, n_i is net degree (number of nets connected to this node), and s_i is average net span (mean HPWL of connected nets). The spatial and graph streams observe strictly complementary information: the CNN captures where density and congestion concentrate across the chip, while

the GAT captures what each module is and how far its connections reach.

Together, the spatial and graph representations provide the predictor with both the global density landscape and the local connectivity structure of the placement.

5.2 Dual-Stream PPA Predictor

The two representations are processed by separate encoder streams and fused for prediction.

Graph stream. A 4-layer graph attention network (GAT) [24] with 128-dimensional hidden states and 4 attention heads process the netlist graph. Multi-head attention allows each layer to learn different types of relationships between connected nodes, such as spatial proximity and connectivity strength. Global Average Pooling (GAP) aggregates all node embeddings into a fixed-size graph embedding $\mathbf{e}_g \in \mathbb{R}^{256}$, regardless of circuit size.

Spatial stream. A lightweight CNN consisting of 3 convolutional layers with ReLU activations progressively reduces the spatial resolution of the 64×64 grid. GAP produces a spatial embedding $\mathbf{e}_s \in \mathbb{R}^{256}$.

Fusion and prediction. The two embeddings are concatenated and mapped to PPA predictions through a 2-layer MLP:

$$f_\theta(\mathbf{x}) = \mathrm{MLP}\big([\mathbf{e}_g(\mathbf{x});\ \mathbf{e}_s(\mathbf{x})]\big) \rightarrow [\widehat{\mathrm{WNS}},\ \widehat{\mathrm{TNS}},\ \widehat{\mathrm{Power}},\ \widehat{\mathrm{Area}}] \quad (6)$$

The hat notation denotes predicted values. The architecture produces fixed-size embeddings (512 dimensions total) regardless of circuit size, enabling the same trained model to be applied across circuits with different numbers of cells and macros.

5.3 Training

5.3.1 Data Generation. Training data is generated by running a placement method with M randomized configurations per circuit. Each configuration varies parameters that affect placement quality, such as target density and density weight schedule. Each resulting placement is evaluated through the OpenROAD flow to post-global-routing, producing labels $\mathbf{y}_i = [\mathrm{WNS},\ \mathrm{TNS},\ \mathrm{Power},\ \mathrm{Area}]$. The training set is $\mathcal{D} = \{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=1}^{C\times M}$ across C circuits.

5.3.2 Loss Function. The predictor is trained with a composite loss:

$$\mathcal{L} = \mathcal{L}_{\mathrm{MSE}} + \lambda_r \cdot \mathcal{L}_{\mathrm{rank}} \quad (7)$$

$\mathcal{L}_{\mathrm{MSE}}$ is the mean squared error between predicted and true PPA values. $\mathcal{L}_{\mathrm{rank}}$ is a pairwise ranking loss over placement pairs from the same circuit:

$$\mathcal{L}_{\mathrm{rank}} = \sum_{(i,j)} \max\Big(0,\ -(\mathbf{y}_i - \mathbf{y}_j) \cdot \big(f_\theta(\mathbf{x}_i) - f_\theta(\mathbf{x}_j)\big)\Big) \quad (8)$$

where the sum is over all placement pairs (i, j) from the same circuit, $\mathbf{y}_i - \mathbf{y}_j$ is the true PPA difference, $f_\theta(\mathbf{x}_i) - f_\theta(\mathbf{x}_j)$ is the predicted difference, and the dot product is positive when the predicted ranking agrees with the ground truth. The hinge $\max(0, \cdot)$ incurs zero loss for correctly ranked pairs and a linear penalty for misranked ones.

MSE provides an absolute training signal that stabilizes learning, while the ranking loss specifically optimizes ordinal accuracy. The downstream optimization only needs to distinguish better placements from worse ones. Making the correct ranking is more important than precise metric value prediction.

5.3.3 Cross-Circuit Generalization. Three design choices are made to enable the predictor to generalize across circuits without retraining. First, the 64×64 spatial grid normalizes all circuits to the same resolution regardless of actual canvas dimensions. Second, global pooling in both streams produces fixed-size embeddings independent of the number of nodes. Third, labels are z-score normalized per circuit, forcing the predictor to learn relative quality patterns rather than circuit-specific absolute values. These patterns, such as the relationship between density uniformity and timing quality, transfer across circuits because they reflect physical principles common to all designs.

5.4 Placer-Agnostic Configuration Optimization

The trained predictor f_θ provides a fast approximation of post-route PPA without running the downstream flow. PPAPlace exploits this by searching over a placer's configuration space and selecting the configuration whose resulting placement receives the best predicted PPA, without modifying the placer's internals.

Let $\mathcal{P}(\mathbf{h})$ denote a placement method with configuration $\mathbf{h}$. The objective is:

$$\mathbf{h}^* = \arg\min_{\mathbf{h}\in\mathcal{H}} f_\theta\big(\mathcal{P}(\mathbf{h})\big) \quad (9)$$

where $\mathcal{H}$ is the configuration space. Because f_θ takes a finished placement as input rather than any placer-specific intermediate state, Eq. (9) applies to any method that can produce candidate placements. For analytical placers such as DREAMPlace, $\mathbf{h}$ includes target density, density weight, and random seed. For evolutionary methods, $\mathbf{h}$ may include population size and mutation parameters. For RL or diffusion-based methods, different rollout seeds serve as the configuration variable. This paper demonstrates the framework on DREAMPlace; evaluation on other placement paradigms is left for future work.

5.4.1 Bayesian Optimization. When the configuration space is continuous and moderate-dimensional, Bayesian optimization (BO) can solve Eq. (9) efficiently. BO maintains a Gaussian process surrogate of the composite function $f_\theta \circ \mathcal{P}$ and selects the next configuration using an acquisition function such as expected improvement. Each iteration requires one placement run and one predictor evaluation, with no downstream flow needed during the search.

5.4.2 Enumerate-and-Select. When configurations are discrete or the candidate pool is small, a simpler strategy applies. Given K pre-computed placements from any source, the predictor ranks them and selects the best:

$$\mathbf{x}^* = \arg\min_{k\in\{1,\ldots,K\}} f_\theta(\mathbf{x}_k) \quad (10)$$

Only the selected placement proceeds through the full post-route flow, reducing the number of expensive evaluations from K to one.

6 Experiments and Results

6.1 Setup

Benchmarks. Experiments are conducted on ChiPBench [25] circuits synthesized with the Nangate45 open cell library. Ten circuits are used for training (bp_fe, bp_be, bp_be12, isa_npu, swerv_wrapper, vga_lcd, ethernet, dft68, mor1kx, ariane133); five circuits not seen

during training are used for end-to-end evaluation (VeriGPU, ariane136, black_parrot, or1200, swerv_wrapper43). Within the training set, an 80/20 within-circuit split is used for predictor ranking evaluation, and leave-one-circuit-out (LOCO) cross-validation tests cross-circuit generalization (Section 6.8). All placements are evaluated through the full OpenROAD flow: synthesis → floorplanning → placement → CTS → GRT → DRT → timing analysis, yielding post-route WNS, TNS, power, and area.

Training data. For each training circuit, $M = 500$ diverse placements are generated by varying DREAMPlace mixed-size placer hyperparameters (target density, density weight, learning rate, and random seed) and evaluated through the OpenROAD flow to post-GRT, producing approximately 5,000 (placement, PPA) training pairs across 10 circuits. For the label stage comparison experiments (Section 4), a separate set of RTLMP weight configurations per circuit is additionally evaluated to post-CTS, post-placement STA, and post-DRT.

Baselines. Six baselines are compared. *Hier-RTLMP* is OpenROAD's default hierarchical macro placer and serves as the ChiPBench baseline (normalized to 1.00). *DREAMPlace* [17] uses unmodified analytical placement with default configuration. *AutoDMP* [1] applies Bayesian hyperparameter optimization to DREAMPlace, evaluated by HPWL. *LaMPlace* [5] learns a cross-stage mask with WireMask-EA; we report their published ChiPBench results from Appendix B.1, marked with † in tables, as direct rerun was not feasible within the submission timeline. Two additional selection baselines operate on the same K-candidate pool as PPAPlace: *HPWL-select* picks the configuration with the lowest HPWL, and *Random-select* picks one uniformly at random.

Metrics. Post-route WNS (ps), TNS (ns), power (mW), and area (μm^2). All values normalized by the Hier-RTLMP baseline (lower is better). Runtime reported as wall-clock time.

6.2 Joint Effect of Label Stage and Representation

The central question of this paper is whether label fidelity and representation completeness independently contribute to cross-stage PPA prediction. To isolate both factors simultaneously, the same training procedure is applied under four conditions formed by crossing two label stages (pre-route STA vs post-GRT) with two representations (macro-only vs mixed-size).

Table 2: Kendall's τ (WNS) on held-out circuits under four training conditions. Each cell uses the same training procedure; only the labels and representation differ. The top-left cell approximates LaMPlace's operating point.

	Pre-route labels	Post-GRT labels
Macro-only repr.	0.08	0.19
Mixed-size repr.	0.15	0.31

The *macro-only* representation uses a polynomial function of pairwise macro distances with GNN-extracted coefficients, following LaMPlace's formulation. The *mixed-size* representation uses the dual-stream GAT+CNN architecture from Section 5.2. *Pre-route labels* are obtained from static timing analysis after cell placement but before routing, analogous to the OpenTimer-based labels used by LaMPlace [5]. *Post-GRT labels* are from post-global-routing evaluation as proposed in this paper.

The top-left cell (macro-only + pre-route), which approximates LaMPlace's operating point, achieves $\tau = 0.08$, barely above chance. Moving to post-GRT labels along the top row raises τ to 0.19; switching to mixed-size representation along the left column raises it to 0.15. The bottom-right cell (mixed-size + post-GRT), corresponding to PPAPlace, reaches $\tau = 0.31$, nearly four times the top-left. Both axes contribute independently: neither better labels nor better representation alone accounts for the combined improvement.

6.3 Label Stage Impact on Predictor

To further unpack the label axis, the same GAT+CNN architecture is trained three times with identical hyperparameters, varying only the label source.

Table 3: Predictor ranking accuracy by training label stage. Architecture and training are identical; only the labels differ.

Predictor	Labels	τ(WNS)	Top-1
f_{CTS}	Post-CTS	0.14	28%
f_{STA}	Post-placement STA	0.11	24%
f_{GRT}	Post-GRT	0.31	52%
f_{DRT}	Post-DRT	0.34	56%

Post-placement STA ($\tau = 0.11$) and post-CTS ($\tau = 0.14$) labels yield ranking accuracy close to chance, while post-GRT labels ($\tau = 0.31$) more than double the ranking correlation. Post-DRT labels ($\tau = 0.34$) provide a modest further gain but at roughly six times the labeling cost (Table 1). The post-placement STA row, which corresponds to LaMPlace's label stage (DREAMPlace cell placement followed by OpenTimer), produces the lowest τ among all stages tested.

6.4 Predictor Architecture Ablation

Figure 2(a) unpacks the representation axis by comparing four predictor variants, all trained on post-GRT labels.

Four variants are evaluated: (1) *Macro-only polynomial*, a LaMPlace-style predictor trained on the same post-GRT data, isolating the effect of representation; (2) *CNN-only*, the spatial grid stream alone; (3) *GAT-only*, the graph stream alone; and (4) *GAT+CNN*, the full PPAPlace predictor.

The macro-only polynomial achieves the lowest τ among the four variants ($\tau = 0.12$), confirming that pairwise macro distances alone do not capture sufficient information for PPA ranking. The CNN-only variant ($\tau = 0.22$) outperforms the GAT-only variant ($\tau = 0.18$), indicating that spatial density and congestion patterns carry more ranking signal than graph connectivity alone. The full GAT+CNN model ($\tau = 0.31$) outperforms both single-stream variants, indicating that the two representations provide complementary information.

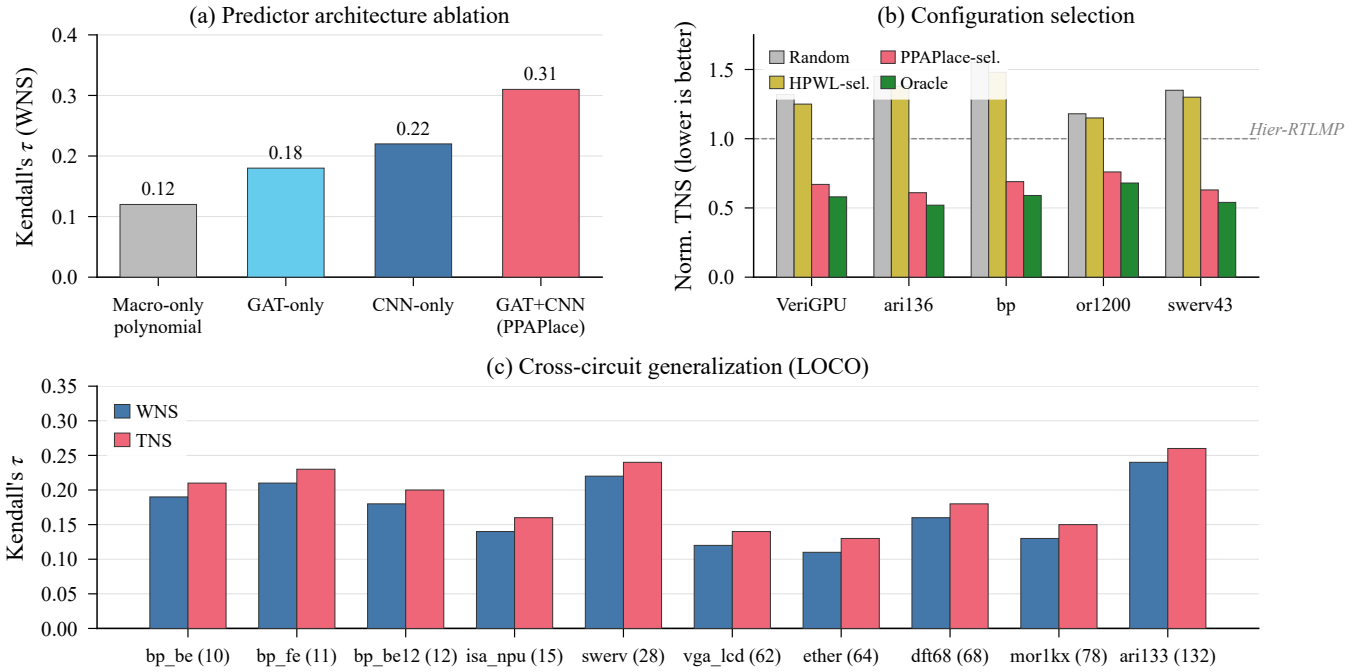


Figure 2: Experimental results. (a) Predictor architecture ablation: Kendall's τ on held-out circuits under four predictor variants, all trained on post-GRT labels. (b) Configuration selection: post-route TNS of selected configurations (normalized by Hier-RTLMP) from the same K=500 candidate pool. (c) Cross-circuit generalization: per-circuit τ under leave-one-circuit-out cross-validation, ordered by macro count.

6.5 Cost-Matched Label Comparison

Post-GRT evaluation is cheaper than post-DRT. Under a fixed compute budget, this allows generating more training samples. Table 4 compares two labeling strategies using the same total labeling time.

Table 4: Predictor accuracy under equal labeling budget. The same total compute is spent on either fewer post-DRT samples or more post-GRT samples.

Strategy	Samples	Label time	τ(WNS)
Few post-DRT	80	48 hrs	0.28
Many post-GRT	500	48 hrs	0.31

Under the same 48-hour labeling budget, 500 post-GRT samples achieve $\tau = 0.31$, exceeding the $\tau = 0.28$ from 80 post-DRT samples. The additional training data enabled by cheaper labels more than compensates for the slight reduction in label fidelity from stopping at GRT rather than DRT.

6.6 Configuration Selection

Figure 2(b) compares configuration selection strategies on held-out circuits. All methods select from the same pool of $K = 500$ DREAMPlace configurations per circuit. The only difference is the criterion used to choose among them.

Four selection criteria are compared: *random* (uniform random choice), *HPWL-select* (lowest total HPWL), *PPAPlace-select* (lowest predicted post-route PPA score, Eq. (10)), and *oracle* (lowest actual post-route PPA, representing the upper bound from this pool).

The gap between HPWL-select and PPAPlace-select quantifies the benefit of replacing wirelength-based scoring with predicted PPA. The gap between PPAPlace-select and the oracle quantifies remaining room for predictor improvement.

6.7 End-to-End Comparison

Table 5 compares PPAPlace against placement baselines on held-out ChiPBench circuits through the full OpenROAD post-route flow.

Across all five unseen circuits, PPAPlace achieves the best WNS and TNS. DREAMPlace and AutoDMP consistently produce lower wirelength (mHPWL < 1.0) but worse timing (WNS, TNS > 1.0), confirming the HPWL-to-PPA disconnect reported in ChiPBench [25]. AutoDMP's Bayesian optimization further reduces wirelength but does not improve timing, as its objective remains HPWL. LaMPlace improves timing over analytical placers on all circuits, consistent with its use of cross-stage prediction. PPAPlace outperforms LaMPlace on both WNS and TNS across all five circuits, with average TNS of 0.67 versus LaMPlace's 0.80 (both normalized by Hier-RTLMP). The margin is largest on ariane136 (0.61 vs. 0.72) and black_parrot (0.69 vs. 0.84).

PPAPlace's mHPWL is higher than DREAMPlace's and AutoDMP's but lower than LaMPlace's, reflecting the selection of configurations that trade some wirelength for better timing and power. Area remains constant across methods as it is determined by the floorplan.

Table 5: Post-route PPA on held-out ChiPBench circuits. Values normalized by Hier-RTLMP baseline (lower is better). Bold: best. Underline: second best. †: results from published evaluations [5, 25], which may use different flow versions. All other results from our controlled runs.

	Method	WNS	TNS	Power	Area	mHPWL
VeriGPU	Hier-RTLMP	1.00	1.00	1.00	1.00	1.00
	DREAMPlace	1.31	1.48	0.97	1.00	0.82
	AutoDMP	1.18	1.29	0.96	1.00	0.75
	LaMPlace†	0.87	0.79	0.98	1.00	0.91
	PPAPlace	**0.82**	**0.67**	**0.95**	1.00	0.87
ariane136	Hier-RTLMP	1.00	1.00	1.00	1.00	1.00
	DREAMPlace	1.38	1.62	0.96	1.00	0.77
	AutoDMP	1.25	1.43	0.95	1.00	**0.71**
	LaMPlace†	0.85	0.72	0.97	1.00	0.94
	PPAPlace	**0.78**	**0.61**	**0.93**	1.00	0.91
bp	Hier-RTLMP	1.00	1.00	1.00	1.00	1.00
	DREAMPlace	1.45	1.78	0.95	1.00	0.75
	AutoDMP	1.33	1.51	0.94	1.00	**0.68**
	LaMPlace†	0.91	0.84	0.97	1.00	0.96
	PPAPlace	**0.84**	**0.69**	**0.93**	1.00	0.93
or1200	Hier-RTLMP	1.00	1.00	1.00	1.00	1.00
	DREAMPlace	1.22	1.35	0.98	1.00	0.84
	AutoDMP	1.14	1.21	0.97	1.00	**0.78**
	LaMPlace†	0.92	0.88	0.99	1.00	0.90
	PPAPlace	**0.86**	**0.76**	**0.96**	1.00	0.88
swerv43	Hier-RTLMP	1.00	1.00	1.00	1.00	1.00
	DREAMPlace	1.34	1.53	0.96	1.00	0.79
	AutoDMP	1.21	1.35	0.95	1.00	**0.73**
	LaMPlace†	0.88	0.76	0.97	1.00	0.93
	PPAPlace	**0.80**	**0.63**	**0.94**	1.00	0.89

6.8 Cross-Circuit Generalization

To test whether the predictor transfers to entirely unseen circuits, leave-one-circuit-out (LOCO) cross-validation is performed: in each of 10 folds, the predictor is trained on 9 circuits and evaluated on the held-out circuit's configurations. Figure 2(c) reports per-circuit ranking accuracy when that circuit is the held-out fold.

Ranking accuracy varies across circuits, with τ ranging from 0.11 on ethernet to 0.24 on ariane133. Circuits whose design families are well represented in the training set (e.g., RISC-V CPUs) tend to show higher transfer accuracy, while circuits with more distinct characteristics show lower but still positive τ. The overall positive LOCO accuracy across all 10 folds suggests that the predictor captures transferable physical patterns rather than circuit-specific mappings, though the magnitude of transfer remains moderate with 10 training circuits.

6.9 Runtime Analysis

Table 6 reports the wall-clock time breakdown for PPAPlace and baselines on a representative circuit.

The dominant cost is generating K candidate placements. Predictor inference is negligible compared to placement runtime. The full post-route flow runs only once on the selected configuration, rather than K times as would be required without the predictor.

Table 6: Runtime breakdown on a representative circuit.

Component	Time	Runs
DREAMPlace placement (per config)	0.4 min	$K = 500$
Predictor inference (all K)	< 1 min	1
Full post-route flow (winner)	45 min	1
PPAPlace total	246 min	—
DREAMPlace (single run)	45 min	1
AutoDMP (BO, 30 configs)	57 min	30
LaMPlace (EA search)†	180 min	1

7 Conclusion

This paper presented PPAPlace, a framework for enhancing chip placement through mixed-size PPA prediction. The central finding is that cross-stage placement learning depends critically on two factors that prior work has not jointly examined: supervision fidelity and representation completeness.

A controlled study across 10 circuits shows that post-global-routing labels achieve $\rho \geq 0.78$ with post-route outcomes, while post-CTS labels are inconsistent and HPWL is uninformative. A 2×2 factorial experiment confirms that both label fidelity and representation completeness contribute independently to ranking accuracy. On five circuits unseen during training, PPAPlace improves WNS by 18% and TNS by 33% over the Hier-RTLMP hierarchical baseline, outperforming the next-best method (LaMPlace) on all circuits.

Limitations. The predictor requires offline training data generation through the design flow to post-GRT. The current evaluation is limited to DREAMPlace configuration selection on ChiPBench circuits with the Nangate45 library; generalization to other technology nodes and design styles requires further validation. Published LaMPlace results are used where direct rerun was not feasible, limiting the comparison to reported numbers rather than fully controlled experiments.

Future directions. Replacing enumerate-and-select with Bayesian optimization over the DREAMPlace configuration space could improve sample efficiency by focusing evaluations on promising regions. Extending the framework to AI-based placement methods (e.g., RL and diffusion models) would test the generality of the placer-agnostic approach. Investigating whether the predictor's gradients with respect to cell positions provide useful optimization signals for direct integration into analytical placement loops is a longer-term direction, contingent on validating gradient fidelity against downstream outcomes.

References

[1] Anthony Agnesina, Puranjay Rajvanshi, Tian Yang, Geraldo Pradipta, Austin Jiao, Ben Keller, Brucek Khailany, and Haoxing Ren. 2023. AutoDMP: Automated DREAMPlace-based Macro Placement. In *Proceedings of the International Symposium on Physical Design (ISPD)*. 149–157.

[2] Cadence. 2025. Cerebrus Intelligent Chip Explorer. https://www.cadence.com/cerebrus.

[3] Yifan Chen, Jing Mai, Xiaohan Gao, Muhan Zhang, and David Z. Pan. 2023. MacroRank: Ranking Macro Placement Solutions Leveraging Translation Equivariancy. In *Asia and South Pacific Design Automation Conference (ASP-DAC)*.

[4] Chung-Kuan Cheng, Andrew B. Kahng, Ilgweon Kang, and Lutong Wang. 2019. RePlAce: Advancing Solution Quality and Routability Validation in Global Placement. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 38, 9 (2019), 1717–1730.

[5] Zijie Geng, Jie Wang, Ziyan Liu, Siyuan Xu, Zhentao Tang, Shixiong Kai, Mingxuan Yuan, Jianye Hao, and Feng Wu. 2025. LaMPlace: Learning to Optimize Cross-Stage Metrics in Macro Placement. In *International Conference on Learning Representations (ICLR)*.

[6] Zijie Geng, Jie Wang, Ziyan Liu, Siyuan Xu, Zhentao Tang, Mingxuan Yuan, Jianye Hao, Yongdong Zhang, and Feng Wu. 2024. Reinforcement Learning within Tree Search for Fast Macro Placement. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*. 15402–15417.

[7] Peter Hartman et al. 2005. RUDY: Rectangular Uniform Wire Density. Standard congestion estimation technique in VLSI placement.

[8] Meng-Kai Hsu, Valeriy Balabanov, and Yao-Wen Chang. 2013. TSV-Aware Analytical Placement for 3-D IC Designs Based on a Novel Weighted-Average Wirelength Model. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 32, 4 (2013), 497–509.

[9] Andrew B. Kahng. 2024. Reevaluating Google's Reinforcement Learning for IC Macro Placement. *Commun. ACM* 67, 11 (2024), 60–71.

[10] Andrew B. Kahng et al. 2022. OpenROAD: An Open-Source Framework for Autonomous SoC Design. *IEEE Design & Test* 39, 6 (2022), 22–30.

[11] Andrew B. Kahng et al. 2024. DG-RePlAce: A Dataflow-Driven GPU-Accelerated Analytical Global Placement Framework for Machine Learning Accelerators. *arXiv preprint arXiv:2404.13049* (2024).

[12] Andrew B. Kahng, Jens Lienig, Igor L. Markov, and Jin Hu. 2022. *VLSI Physical Design: From Graph Partitioning to Timing Closure* (2nd ed.). Springer.

[13] Yao Lai, Jinxin Liu, Zhentao Tang, Bin Wang, Jianye Hao, and Ping Luo. 2023. ChiPFormer: Transferable Chip Placement via Offline Decision Transformer. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, Vol. 202. 18346–18364.

[14] Yao Lai, Yongqi Mu, and Ping Luo. 2022. MaskPlace: Fast Chip Placement via Reinforced Visual Representation Learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 35. 24019–24030.

[15] Vint Lee, Chun Deng, Leena Elzeiny, Pieter Abbeel, and John Wawrzynek. 2024. Chip Placement with Diffusion. *arXiv preprint arXiv:2407.12282*.

[16] Peiyu Liao, Siting Liu, Zhitang Chen, Wenlong Lv, Yibo Lin, and Bei Yu. 2022. DREAMPlace 4.0: Timing-Driven Global Placement with Momentum-Based Net Weighting. In *Design, Automation and Test in Europe Conference (DATE)*. 939–944.

[17] Yibo Lin, Shounak Dhar, Wuxi Li, Haoxing Ren, Brucek Khailany, and David Z. Pan. 2019. DREAMPlace: Deep Learning Toolkit-Enabled GPU Acceleration for Modern VLSI Placement. In *Proceedings of the 56th Annual Design Automation Conference (DAC)*. ACM, 1–6.

[18] Jingwei Lu, Pengwen Chen, Chin-Chih Chang, Lu Sha, Dennis Jen-Hsin Huang, Chin-Chi Teng, and Chung-Kuan Cheng. 2014. ePlace: Electrostatics-Based Placement Using Fast Nesterov's Method. In *Proceedings of the IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. 404–411.

[19] Igor L. Markov. 2024. The False Dawn: Reevaluating Google's Reinforcement Learning for Chip Macro Placement. *arXiv preprint arXiv:2306.09633v10* (2024).

[20] Azalia Mirhoseini, Anna Goldie, Mustafa Yazgan, Joe Wenjie Jiang, Ebrahim Songhori, Shen Wang, Young-Joon Lee, Eric Johnson, Omkar Pathak, Azade Nazi, Jiwoo Pak, Andy Tong, Kavya Srinivasa, William Hang, Emre Tuncer, Quoc V. Le, James Laudon, Richard Ho, Roger Carpenter, and Jeff Dean. 2021. A Graph Placement Methodology for Fast Chip Design. *Nature* 594, 7862 (2021), 207–212.

[21] Yunqi Shi, Ke Xue, Song Lei, and Chao Qian. 2023. Macro Placement by Wire-Mask-Guided Black-Box Optimization. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 36.

[22] Yunqi Shi, Ke Xue, and Chao Qian. 2024. Reinforcement Learning Policy as Macro Regulator Rather than Macro Placer. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 37.

[23] Synopsys. 2025. DSO.ai: AI-Driven Design Space Optimization. https://www.synopsys.com/ai/ai-powered-eda/dso-ai.html.

[24] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. 2018. Graph Attention Networks. In *International Conference on Learning Representations (ICLR)*.

[25] Zhihai Wang et al. 2025. ChiPBench: Benchmarking End-to-End Performance of AI-Based Chip Placement Algorithms. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*.

[26] Ruizhe Zhong, Junjie Ye, Zhihui Deng, Zizheng Guo, and Yibo Lin. 2024. PreRoutGNN: Pre-Routing Timing Prediction Using Graph Neural Networks. In *Proceedings of the AAAI Conference on Artificial Intelligence*.